

Experiment - 1

- **Aim** - Introduction to MY SQL
- **Objective** - To understand about different SQL command and keys
- **Theory** –

Oracle has many tools such as SQL * PLUS, Oracle Forms, Oracle Report Writer, Oracle Graphics etc.

- ❖ **SQL * PLUS:** The SQL * PLUS tool is made up of two distinct parts. These are
- ❖ **Interactive SQL:** Interactive SQL is designed for create, access and manipulate data structures like tables and indexes.
- ❖ **PL/SQL:** PL/SQL can be used to developed programs for different applications.
- ❖ **Oracle Forms:** This tool allows you to create a data entry screen along with the suitable menu objects. Thus it is the oracle forms tool that handles data gathering and data validation in a commercial application.
- ❖ **Report Writer:** Report writer allows programmers to prepare innovative reports using data from the oracle structures like tables, views etc. It is the report writer tool that handles the reporting section of commercial application.
- ❖ **Oracle Graphics:** Some of the data can be better represented in the form of pictures. The oracle graphics tool allows programmers to prepare graphs using data from oracle structures like tables, views etc.

❖ What is My SQL?

MySQL can be described as an open-source relational database management system (RDBMS) based on Structured Query Language (SQL). It is a hugely popular database used throughout the web — from large multinationals to simple sites, and databases of all shapes and sizes. MySQL was originally developed by a Swedish company called MySQL AB in 1995. Now, Oracle Corporation owns it. MySQL is designed to read, create, update, and delete data from databases that are structured as a table (relational format) where the tables have defined relationships among themselves with keys and indexes. It is widely used with server-side programming languages like PHP, Python & Java in web development.

❖ History of My SQL

Original authors: Michael Widenius, David Axmark, and Allan Larsson (initial release: April 1995). The system was created initially because there were no solutions for databases that were lightweight and fast. Since then, MySQL has matured through the efforts of a broad community of developers around the world and ultimately became the technology upon which many web applications are founded today, particularly in the LAMP stack (Linux, Apache, MySQL, PHP/Python/Perl). MySQL was acquired by Sun Microsystems in 2008, and then Oracle Corporation bought Sun Microsystems in 2010, which put in question the future development of MySQL. MySQL is still one of the most commonly used DBMS today, especially in open-source communities and web-based applications.

[Type here]

❖ DATA TYPES:

- 1. CHAR (Size):** This data type is used to store character strings values of fixed length. The size in brackets determines the number of characters the cell can hold. The maximum number of character is 255 characters.
- 2. VARCHAR (Size) / VARCHAR2 (Size):** This data type is used to store variable length alphanumeric data. The maximum character can hold is 2000 character.
- 3. NUMBER (P, S):** The NUMBER data type is used to store number (fixed or floating point). Number of virtually any magnitude may be stored up to 38 digits of precision. Number as large as $9.99 * 10^{124}$. The precision (p) determines the number of places to the right of the decimal. If scale is omitted then the default is zero. If precision is omitted, values are stored with their original precision up to the maximum of 38 digits.
- 4. DATE:** This data type is used to represent date and time. The standard format is DD-MM-YY as in 17-SEP-2009. To enter dates other than the standard format, use the appropriate functions. Date time stores date in the 24-Hours format. By default the time in a date field is 12:00:00 am, if no time portion is specified. The default date for a date field is the first day the current month.
- 5. LONG:** This data type is used to store variable length character strings containing up to 2GB. Long data can be used to store arrays of binary data in ASCII format. LONG values cannot be indexed, and the normal character functions such as SUBSTR cannot be applied.
- 6. RAW:** The RAW data type is used to store binary data, such as digitized picture or image. Data loaded into columns of these data types are stored without any further conversion. RAW data type can have a maximum length of 255 bytes. LONG RAW data type can contain up to 2GB.

❖ Basic SQL Commands in My SQL

1. Data Definition Language (DDL)

- DDL changes the structure of the table like creating a table, deleting a table, altering a table, etc.
- All the command of DDL are auto-committed that means it permanently save all the changes in the database.

Following are the some commands that come under DDL:

1. CREATE: This is used to create a new relation (table)

Syntax:

CREATE TABLE TABLE_NAME (COLUMN_NAMES DATATYPES [...]);

Example:

CREATE TABLE EMPLOYEE(Name VARCHAR2(20), Email VARCHAR2(100), DOB DATE);

2. DROP: It is used to delete both the structure and record stored in the table.

Syntax:

DROP TABLE table_name [cascade constraint];

Example:

```
DROP TABLE EMPLOYEE;
```

3. **ALTER**: It is used to alter the structure of the database. This change could be either to modify the characteristics of an existing attribute or probably to add a new attribute.

a)Syntax: To add a new column in the table

```
ALTER TABLE table_name ADD column_name column-definition;
```

Example:

```
ALTER TABLE STU_DETAILS ADD (ADHAR_NUM VARCHAR2 (15));
```

b)Syntax: To ADD a multiple column from a table..

```
ALTER TABLE table_name ADD column_name1, column_name2;
```

Example:

```
ALTER TABLE STU_DETAILS ADD ADHAR_NUM, NAME;
```

c) Syntax: To ADD a constraint from a table.

```
ALTER TABLE table_name ADD (column_name column-  
definition CONSTRAINT constraint_name);
```

Example:

```
ALTER TABLE STU_DETAILS ADD (CONSTRAINT PK_STU_DETAILS PRIMARY KEY  
(STU_ID);
```

3. **DROP TABLE**: This is used to delete the structure of a relation. It permanently deletes the records in the table.

Syntax: DROP TABLE relation_name;

Example: SQL>DROP TABLE std;

4. **RENAME**: It is used to modify the name of the existing database object.

Syntax:

```
RENAME TABLE old_relation_name TO new_relation_name;
```

Example:

```
SQL>RENAME TABLE std TO std1;
```

2. Data Manipulation Language

- DML commands are used to modify the database. It is responsible for all form of changes in the database.
- The command of DML is not auto-committed that means it can't permanently save all the changes in the database. They can be rollback.

Following are the some commands that come under DML:

1. **UPDATE**: To update a record in a table you use the UPDATE statement. Use the WHERE condition to specify which records you want to update. It is possible to update one or more columns at a time.

Syntax: UPDATE table_name

```
SET column1 = value1,
```

```
column2 = value2, ...
```

```
WHERE condition;
```

Example:

```
UPDATE Person
```

```
[Type here]
```

SET Name = "Elton John"

WHERE Id = 4;

2. INSERT: The INSERT statement is a SQL query. It is used to insert data into the row of a table. To insert a new row into a table you must be your on schema or INSERT privilege on the table.

Syntax:

INSERT INTO TABLE_NAME (col1, col2, col3,.... col N) VALUES (value1, value2, value3, valueN);

Example:

INSERT INTO javatpoint (Author, Subject) VALUES ("HS", "DBMS");

3. DELETE: It is used to remove one or more row from a table. To delete rows from the table, it must be in your schema or you must have delete privilege.

Syntax:

DELETE FROM table_name [WHERE condition];

Example:

DELETE FROM javatpoint
WHERE Author="HS";

4. SELECT: This is the same as the projection operation of relational algebra. It is used to select the attribute based on the condition described by WHERE clause.

Syntax:

SELECT expressions
FROM TABLES
WHERE conditions;

Example:

SELECT emp_name
FROM employee
WHERE age > 20;

- **Conclusion:**

MySQL is a fast, reliable, and open-source server mechanism, well-suited for small to large-scale enterprise applications. Due to its speed, security, and reliability, it is one of the most widely used databases globally, especially in web development. Developers will be able to use MySQL effectively with their applications and provide proper data management and integrity by learning the features of MySQL, installation processes, as well as carrying out basic SQL operations. Once you become well-versed with MySQL, you can explore more advanced topics like indexing, transactions, and replication to unlock its full potential

EXPERIMENT – 2

AIM: To study basic SQL commands (create database, create table, use , drop, insert) and execute the queries using these commands.

Create database: The create database is used to create a new SQL database.

Syntax: CREATE DATABASE <DATABASE_NAME>; **Example:**

create database world;

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| world |
+-----+
```

Use Database: This database tells MYSQL to use the named database as the Default (current) database for subsequent statements.

```
mysql> use world;
Database changed
```

Create Table: The table consist of rows and columns. The columns specify the type of data, whereas the rows contain the actual data itself.

Syntax: CREATE TABLE <relation_name/table _name>

(field_1 data_type(size), field_2 data_type(size), ..)

Example: create table Shashwat(ID int Primary Key, Name varchar(20), Address varchar(20));

```
mysql> desc Shashwat;
```

Field	Type	Null	Key	Default	Extra
ID	int	NO	PRI	NULL	
Name	varchar(20)	YES		NULL	
Address	varchar(20)	YES		NULL	

INSERT INTO TABLE: The insert statement is used to insert new records in a table.

SYNTAX: insert into <table_name> ("values")

```
mysql> insert into Shashwat(ID, Name, Address) Values
-> (1, "Shashwat", "Lucknow"),
-> (2, "Tarun", "Tundla"),
-> (3, "Saksham", "Delhi"),
-> (4, "Aditya", "Delhi"),
-> (5, "Shreyas", "Delhi"),
-> (6, "Suresh", "Delhi"),
-> (7, "Rudransh", "Mainpuri"),
-> (8, "Aaryan", "Hisar"),
-> (9, "Bhavya", "Jaipur"),
-> (10, "Raghuvansh", "Kanpur");
Query OK, 10 rows affected (0.08 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Drop database and table: Drop table delete the table and drop database deletes the database that you have created.

SYNTAX: drop database<database_name>;

EXAMPLE: drop database world;

```
+-----+
| Database |
+-----+
| city     |
| company  |
| information_schema |
| mysql     |
| performance_schema |
| sales     |
| sys      |
+-----+
7 rows in set (0.02 sec)
```

EXPERIMENT – 3

AIM: To study the viewing commands (select*, select) , use of comparison operator (>,<,>=,<=,=) and execute the queries using these commands.

Select*: select* statement is used to viewing the values we have inserted in the table.

Syntax: select*from<table_name>;

Example: select* from Shashwat;

```
mysql> select * from Shashwat;
+----+-----+-----+
| ID | Name   | Address |
+----+-----+-----+
| 1  | Shashwat | Lucknow |
| 2  | Tarun   | Tundla  |
| 3  | Saksham | Delhi   |
| 4  | Aditya  | Delhi   |
| 5  | Shreyas | Delhi   |
| 6  | Suresh  | Delhi   |
| 7  | Rudransh | Mainpuri |
| 8  | Aaryan  | Hisar   |
| 9  | Bhavya  | Jaipur  |
| 10 | Raghuvarsh | Kanpur |
+----+-----+-----+
10 rows in set (0.00 sec)
```

Select: select is the keyword used to retrieve the data. We can specify one or more columns separated by commas.

Syntax: select column_1, column_2 from <table_name>;

Example: select ID, Address from Shashwat;


```
mysql> select ID, Address from Shashwat;
+----+-----+
| ID | Address |
+----+-----+
| 1  | Lucknow |
| 2  | Tundla  |
| 3  | Delhi   |
| 4  | Delhi   |
| 5  | Delhi   |
| 6  | Delhi   |
| 7  | Mainpuri|
| 8  | Hisar   |
| 9  | Jaipur  |
| 10 | Kanpur  |
+----+-----+
10 rows in set (0.00 sec)
```

COMPARISON OPERATOR: In MYSQL, the comparison operator are used to compare values in a WHERE clause to determine the truth of the condition. These operators allow you to compare two values and return result based on the comparison.

1. EQUAL TO: it return the row/tuple s which have the value of the attribute equal to the given value.

Example: select* from Shashwat where ID = 5;

```
mysql> select * from Shashwat where ID = 5;
+----+-----+-----+
| ID | Name   | Address |
+----+-----+-----+
| 5  | Shreyas | Delhi   |
+----+-----+-----+
```

2. GREATER THAN(>): it return the row/tuple s which have the value of the attribute greater than the given value.

Example: select* from Shashwat where ID > 5;

```
mysql> select * from Shashwat where ID > 5;
+----+-----+-----+
| ID | Name       | Address |
+----+-----+-----+
| 6  | Suresh     | Delhi   |
| 7  | Rudransh   | Mainpuri|
| 8  | Aaryan     | Hisar   |
| 9  | Bhavya     | Jaipur  |
| 10 | Raghuvansh | Kanpur  |
+----+-----+-----+
5 rows in set (0.00 sec)
```

3. LESS THAN(<): it return the row/tuple s which have the value of the attribute less than the given value.

Example: select* from Shashwat where ID < 5;

```
mysql> select * from Shashwat where ID < 5;
```

ID	Name	Address
1	Shashwat	Lucknow
2	Tarun	Tundla
3	Saksham	Delhi
4	Aditya	Delhi

4 rows in set (0.00 sec)

4. GREATER THAN OR EQUAL TO(>=): it returns the row/tuple s which have the value of the attribute greater or equal to the given value.

Example: select* from Shashwat where ID >= 5;

```
mysql> select * from Shashwat where ID >= 5;
```

ID	Name	Address
5	Shreyas	Delhi
6	Suresh	Delhi
7	Rudransh	Mainpuri
8	Aaryan	Hisar
9	Bhavya	Jaipur
10	Raghuvansh	Kanpur

6 rows in set (0.00 sec)

5. LESS THAN OR EQUAL TO(<=): it returns the row/tuple s which have the value of the attribute less or equal to the given value.

Example: select* from Shashwat where ID <= 5;

```
mysql> select * from Shashwat where ID <= 5;
```

ID	Name	Address
1	Shashwat	Lucknow
2	Tarun	Tundla
3	Saksham	Delhi
4	Aditya	Delhi
5	Shreyas	Delhi

6. NOT EQUAL TO(!=): it returns the row/tuple s which have the value of the attribute that is not equal to the given value.

Example: select* from Shashwat where ID != 5;

```
mysql> select * from Shashwat where ID != 5;
```

ID	Name	Address
1	Shashwat	Lucknow
2	Tarun	Tundla
3	Saksham	Delhi
4	Aditya	Delhi
6	Suresh	Delhi
7	Rudransh	Mainpuri
8	Aaryan	Hisar
9	Bhavya	Jaipur
10	Raghuvansh	Kanpur

9 rows in set (0.00 sec)

Experiment - 4

Aim - To study the commands to modify the structure of the table (alter, drop, modify, update, like) and execute the queries using these commands.

1. ALTER TABLE: Used to add, delete, or modify columns in an existing table. **A) Add a column:**

Syntax: ALTER TABLE

table_name ADD

column_name datatype;

Example:

```
mysql> alter table Shashwat add column Course Varchar(20);
Query OK, 0 rows affected (0.24 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> select * from Shashwat;
+----+-----+-----+-----+
| ID | Name   | Address | Course |
+----+-----+-----+-----+
| 1  | Shashwat | Lucknow | NULL   |
| 2  | Tarun   | Tundla  | NULL   |
| 3  | Saksham | Delhi   | NULL   |
| 4  | Aditya  | Delhi   | NULL   |
| 5  | Shreyas | Delhi   | NULL   |
| 6  | Suresh  | Delhi   | NULL   |
| 7  | Rudransh | Mainpuri | NULL   |
| 8  | Aaryan  | Hisar   | NULL   |
| 9  | Aditya  | Jaipur  | NULL   |
| 10 | Raghuvarsh | Kanpur | NULL   |
+----+-----+-----+-----+
10 rows in set (0.00 sec)
```

B) Drop a column:

Syntax: ALTER TABLE

table_name DROP

COLUMN column_name;

Example:

```
mysql> alter table Shashwat drop column course;
Query OK, 0 rows affected (0.22 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> select * from Shashwat;
+----+-----+-----+
| ID | Name      | Address |
+----+-----+-----+
| 1  | Shashwat  | Lucknow |
| 2  | Tarun     | Tundla  |
| 3  | Saksham   | Delhi   |
| 4  | Aditya    | Delhi   |
| 5  | Shreyas   | Delhi   |
| 6  | Suresh    | Delhi   |
| 7  | Rudransh  | Mainpuri |
| 8  | Aaryan    | Hisar   |
| 9  | Aditya    | Jaipur  |
| 10 | Raghuvansh | Kanpur  |
+----+-----+-----+
10 rows in set (0.02 sec)
```

C) Modify a column:

Syntax: ALTER TABLE table_name

MODIFY COLUMN column_name

datatype; **Example:**

```
mysql> alter table Shashwat modify column Course char;
Query OK, 10 rows affected (2.96 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> desc Shashwat;
+----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+----+-----+-----+-----+-----+-----+
| ID    | int           | NO   | PRI | NULL    |       |
| Name  | varchar(20)   | YES  |     | NULL    |       |
| Address | varchar(20)  | YES  |     | NULL    |       |
| Course | char(1)       | YES  |     | NULL    |       |
+----+-----+-----+-----+-----+-----+
4 rows in set (1.04 sec)
```

2. UPDATE TABLE: Used to update records in an existing table.

Syntax: UPDATE TABLE table_name set name = "NAME" where name = "Name";

```
mysql> update Shashwat Set name = "Aditya" where name = "Bhavya";
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select name from Shashwat;
+-----+
| name |
+-----+
| Shashwat |
| Tarun |
| Saksham |
| Aditya |
| Shreyas |
| Suresh |
| Rudransh |
| Aaryan |
| Aditya |
| Raghuvansh |
+-----+
10 rows in set (0.00 sec)
```

3. LIKE OPERATOR: the LIKE operator is used in a WHERE clause to search for a specified pattern in a column. It is often used with wildcard characters to match patterns.

```
mysql> select name from Shashwat where Name like 'S__%';
+-----+
| name |
+-----+
| Shashwat |
| Saksham |
| Shreyas |
| Suresh |
+-----+
4 rows in set (0.00 sec)
```

```
mysql> select name from Shashwat where Name like '%Shashwat%';
```

```
+-----+  
| name   |  
+-----+  
| Shashwat |  
+-----+
```

```
1 row in set (0.01 sec)
```

```
mysql> select name from Shashwat where Name like '%S';
```

```
+-----+  
| name   |  
+-----+  
| Shreyas |  
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> select name from Shashwat where Name like 'S%';
```

```
+-----+  
| name   |  
+-----+  
| Shashwat |  
| Saksham  |  
| Shreyas  |  
| Suresh   |  
+-----+
```

```
4 rows in set (0.00 sec)
```

```
mysql> select name from Shashwat where Name like 'S%M';
```

```
+-----+  
| name   |  
+-----+  
| Saksham |  
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> select name from Shashwat where Name like '_A%M';
```

```
+-----+  
| name   |  
+-----+  
| Saksham |  
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> select name from Shashwat where Name like 'S_%';
```

```
+-----+  
| name   |  
+-----+  
| Shashwat |  
| Saksham  |  
| Shreyas  |  
| Suresh   |  
+-----+
```